

Prism — Prompts, System Instructions & Workflows

This document inventories every AI prompt, system instruction, and non-AI workflow used to build and run Prism (Kohler-MITWPU AI Research Lab, Track 3). Export this Markdown to PDF for the submission requirement.

Where to look in a live demo: each chat reply returns `workflow + prompt_doc` in the API (`ChatResponse`) and shows them under the bubble as “Prompt / workflow”. Source URLs under **Sources** are the retrieved chunks that grounded the answer (citation integrity drops extras).

Sourcing split (stated explicitly): Customer Support, Privacy, and Legal/Compliance are **real** public Kohler documents. HR and Finance are **synthetic** Meridian Fixtures policies, authored because no real internal Kohler HR/Finance manuals are public. Full rationale: `docs/decisions.md §6`.

Workflow ids returned by the API (match the UI chip):

workflow	Meaning	See
<code>rag_canonical</code>	Hybrid retrieve → SYSTEM_PROMPT JSON answer → prose	§1
<code>rag_email</code>	Same RAG path, then email renderer	§1, §3
<code>policy_math</code>	Deterministic leave / approval-band arithmetic	§4
<code>rbac_deny</code>	Role denied before retrieval	§4 RBAC
<code>jailbreak_refuse / policy_override_refuse</code>	Deterministic refuse	§4
<code>reformat</code>	Re-render <code>last_answer</code> only	§4
<code>clarify</code>	Ambiguous domain → clarifying question	§4
<code>session_memory / session_email_recall</code>	History lookback	§4
<code>upload_rag</code>	Session-scoped uploaded document	§4

1. Runtime system prompt — answer generation

File: prism/core/answer.py → SYSTEM_PROMPT

Model: PRISM_GEN_MODEL (default qwen2.5:7b-instruct)

When: Once per new user question (not on reformat-only turns).

Output contract: A single JSON object matching CanonicalAnswer (Piece 8). All five UI formats are pure-Python renderers over that object.

```
You are Prism, Kohler's internal enterprise knowledge assistant.
You answer questions about HR policy, Finance policy, Customer Support
(product troubleshooting), Privacy, and Legal/Compliance using ONLY the
provided context chunks.
```

Rules:

- Ground every factual claim in the provided context. Do not invent numbers, dates, model numbers, or policy terms.
- If the context does not contain a confident answer, set "no_answer": true and explain what's missing in "direct_answer" -- never fabricate a plausible-sounding answer, especially for Legal/Privacy questions.
- Copy numeric values (currency, percentages, day counts, thresholds) EXACTLY as they appear in the context. Do not round or approximate.
- Populate "key_facts" with the specific facts/values the user needs.
- Populate "steps" only if the answer is a procedure. Use [] (empty array), never null, when there are no steps.
- Populate "table" only if the answer is genuinely tabular -- otherwise null.
- When matching a numeric amount to a threshold band, pick the band that CONTAINS that amount. Do not pick a higher band.
- "sources" must list the source_url of every context chunk you actually used.
- Keep "direct_answer" as plain prose with no markdown formatting.
- Output ONLY valid JSON matching the provided schema. Use empty arrays [] (not null) for key_facts, steps, caveats, and sources when empty.

User prompt template (same file, _build_user_prompt): injects domain, recent conversation context, the user question, retrieved chunks formatted as [Source N] title=... url=... domain=..., and the JSON schema.

Repair retry: On JSON/schema validation failure, one follow-up user message is appended with the pydantic/JSON error and a request for corrected JSON only. After a second failure, the agent returns a safe no_answer object instead of crashing.

2. Session titling prompts

File: prism/core/titler.py

Model: PRISM_TITLE_MODEL (default qwen2.5:3b-instruct) — cheap auxiliary model, not the generation model.

When: Once after the first successful exchange; optionally again on a clear domain switch (maybe_retitle).

Typical instruction pattern: - Produce a short (≤ 6 word) descriptive session title. - No quotes, no trailing punctuation, no "Chat about...". - Reflect the user's topic, not the assistant's wording.

3. Email renderer (optional polish)

File: `prism/core/renderers/email.py`

Default path is **template-only** (no LLM): subject + greeting + body assembled from `CanonicalAnswer`.

An optional polish pass can be enabled for a more natural register; it is **off by default** so reformatting never burns an extra LLM call during demos.

4. Non-LLM “prompts” / decision rules (intentionally not model calls)

These are documented here because they are part of the agent workflow even though they are not LLM prompts — they exist specifically to protect the 8GB VRAM budget.

Workflow	Mechanism	File
Domain routing	Cosine similarity of query embedding vs. ~ 12 hand-written anchor phrases per domain + domain vote over top hybrid hits	<code>prism/core/router.py</code>
Ambiguity $\rightarrow$ clarification	If top-two domain scores within <code>ROUTER_AMBIGUITY_MARGIN</code> (0.04), ask a clarifying question instead of guessing	<code>prism/core/agent.py</code>
No-answer honesty	If best dense distance exceeds relevance floor (stricter for Legal/Privacy), skip LLM and return grounded gap	<code>prism/core/retriever.py</code> , <code>answer.no_context_answer</code>
Reformat request	Regex detect “as JSON / XML / excel / email” $\rightarrow$ render <code>last_answer</code> only	<code>prism/core/agent.py</code>

Workflow	Mechanism	File
Anaphora	Fold recent turn text into retrieval query when “that/it/the one...” appears; plus one extra top_k=3 “focus” retrieval on the raw follow-up so a topic pivot (“...covered under warranty”, “...refund instead”) is not drowned by the previous topic, and a generation note to answer the follow-up rather than re-summarize	prism/core/agent.py

Workflow	Mechanism	File
Session document upload (ad-hoc 6th domain)	POST /sessions/{id}/upload → parse by extension (pypdf / python-docx / BeautifulSoup / text) → heading-aware chunk at 450 tokens → embed with the same nomic-embed-text → separate Chroma collection prism_uploads tagged session_id/doc_id. Retrieval is filtered by session; used when the user points at the file (“this document”, filename) or upload confidence passes a dense floor and a comparative check against the KBs (UPLOAD_KB_MARGIN). Deterministic policy_math is skipped when the user points at their file so Meridian numbers never override the user’s own document. Purged on session delete, on remove, and by a 24h TTL sweep at startup. No new prompt — the generation call gets an extra note: <i>“Answer ONLY from those uploaded chunks... if the uploaded text does not answer, set no_answer=true.”</i>	prism/core/uploads.py, agent.py, api/routes.py
Jailbreak / prompt exfil	Deterministic refuse (no LLM) on “ignore previous instructions / print system prompt / developer mode”	prism/core/text_utils.py, agent.py

Workflow	Mechanism	File
RBAC (email + role)	bcrypt-hashed SQLite users; opaque Bearer tokens; Chroma where ANDs <code>role_{role}=True</code> before any chunk reaches the LLM (not a UI-only gate). Customers → public Support/Privacy/Legal only. General employees → + HR/Finance policy. HR staff → + employee records. Finance staff → + compensation/CTC. Denials logged to <code>access_denials</code> and shown to staff. Chat claims of authority never change role.	<code>prism/core/rbac.py</code>, <code>auth.py</code>, <code>retriever.py</code>, <code>agent.py</code>
Conflicting sources	Prefer official PDF/DOCX (<code>source_priority=100</code>) over scraped HTML (10). Sort retrieved chunks by priority into the prompt; append a caveat naming the secondary Assist article. Users get one confident answer with a footnote — not a dump of two manuals.	<code>answer_polish.apply_source_priority_caveat</code> <code>warranty_fixture</code> <code>legal_warranty_fixture.jsonl</code>
Authority spoof / policy override	Deterministic refuse on CFO-spoof + “override threshold to Rs.0” (same path every time — not model-dependent)	<code>prism/core/text_utils.py</code>, <code>agent.py</code>
Leave / Finance arithmetic	Code computes CL carry-forward, leave-year join-date math, and Finance §2.1 approval bands from a stated claim amount	<code>prism/core/policy_math.py</code>

Workflow	Mechanism	File
Water-conservation estimate	When a Customer Support answer is about a leak / drip / running toilet, append sustainability_note from EPA WaterSense / Fix-a-Leak published figures (31 L/day drip; 757 L/day running toilet). Never an LLM call; shown as a UI callout	prism/core/water_math.py
Output constraints	Detect “yes or no only” / “one word only”; post-polish collapses the answer to that form (Excel then correctly declines)	text_utils.py, answer_polish.py
Soft adversarial paraphrases	VIP waiver / hidden-rules asks that bypass regex → LLM + polish still refuse	text_utils.py, answer_polish.py
Status streaming	SSE stages (routing / retrieving / generating) then full ChatResponse — no partial answer tokens	api/routes.py, frontend chatStream
Citation integrity	Drop answer sources that are not among retrieved chunk source_urls	answer_polish.py
Ollama keep-alive	Shared client; keep_alive on chat/embed; warm models on API startup	ollama_client.py, api/main.py
Session lookback	“What did I ask two questions ago?” answered from history (no RAG)	agent.py
Long-range email recall	“Draft email about leave carry-forward from earlier” rebuilds the CL=5 fact from session, then email-renders	agent.py

Workflow	Mechanism	File
Leading-number sycophancy	Detect “my manager said Rs.X... isn’t it?”; instruct + post-polish so we never rubber-stamp fabricated figures	text_utils.py, answer_polish.py
Multi-domain retrieve	Per-domain retrieve + merge (top_k=3, fuse≤6); skip agnostic pass when domain signals are clear; filter Bathroom Design Service noise	agent.py
Retrieval confidence	Dense distance under floor or strong BM25 rank / dual-signal RRF (exact Rs. / model tokens)	retriever.py
BM25 sidecar	Stores ids + docs + metadatas so sparse hits need no per-id Chroma get	store.py

Anchor phrases are literal English examples (e.g. HR: “leave policy”, “WFH”, “notice period”; Finance: “expense reimbursement”, “per diem”; Customer Support: “toilet leaking”, “faucet aerator”; Privacy: “personal information”, “Do Not Sell”; Legal: “terms and conditions”, “Prop 65”, “warranty”). Full list lives in router.py.

5. Workflows used to *build* the knowledge base (not runtime chat)

5a. Assist crawl + parse

1. Fetch <https://assist.kohler.com/robots.txt> — Allow:/, Disallow:/_next/.
2. Parse <https://assist.kohler.com/en/sitemap> for all /en/<category>/<slug> links.
3. Polite fetch (~1 req/sec, cached under data/raw/assist/).
4. Parse __NEXT_DATA__ → displayCard (title, markdown details, choices) — not rendered HTML — to avoid nav/footer noise.
5. Classify type: troubleshooting / install-guide / video / warranty / policy.
6. Deduplicate cross-listed slugs; tag all categories.
7. One article ≈ one chunk.

5b. Kohler.com legal / privacy

1. curl_cffi with Chrome TLS impersonation (Akamai blocks bare httpx).

2. Heading-boundary chunking on Privacy Policy and Terms.
3. Prop 65 / SDS / green-building / warranty pages from Assist cross-tagged into Legal.

5c. Synthetic HR & Finance authoring

- Written as Markdown by the submitting student (not by an LLM “generate a policy” prompt used as the final artifact).
- Structure informed by reference PDFs (BU_HR_Manual_.pdf, Sample_Financial_Policies_AFOA.pdf) for section conventions only.
- Clear SYNTHETIC disclosure banner in each file.
- Finance deliberately encodes unambiguous Rs. thresholds for exactness demos.
- Chunked by heading via `prism/ingest/build_synthetic.py`.

5d. Index build

- One Chroma collection `prism_kb` with `domain + boolean domain_*` flags.
- BM25 sidecar for exact tokens (model numbers, Rs. amounts).
- Embeddings: `nomic-embed-text` with on-disk cache.

6. Eval / benchmark prompts

`eval/bench_models.py` and `eval/run_eval.py` reuse the same `SYSTEM_PROMPT + generate_answer()` path as production — there is no separate “eval prompt.” Golden questions live in `eval/golden.jsonl`.

7. UI copy (non-model)

Empty-state headline and suggestion chips in `frontend/src/App.tsx` are static product copy, not LLM-generated. The upload affordance (attach button, drag-and-drop, “active” document chips, “Local only · purged with this chat” note) is likewise static copy.

Keep this file updated whenever a prompt or workflow changes; regenerate the PDF before final submission.